

Universidade Federal de São Carlos
Centro de Ciências Exatas e de Tecnologia
Departamento de Engenharia Elétrica

Giovanni Aurelio

Previsão de Emissões de CO₂ de Usinas Termelétricas Nacionais
utilizando Redes Neurais Artificiais

São Carlos
2026

Giovanni Aurelio

Previsão de Emissões de CO₂ de Usinas Termelétricas Nacionais utilizando Redes Neurais Artificiais

Projeto de monografia de conclusão de curso apresentado ao Curso de Engenharia Elétrica da Universidade Federal de São Carlos como parte dos requisitos necessários para a obtenção do grau de Engenheiro Eletricista.

Orientador: Prof. Dr. Guilherme Guimarães Lage

São Carlos
2026

Dedico este trabalho a todos que me ajudaram a superar os desafios, em especial, os meus colegas da turma Elétrica 019. Sem vocês, tudo seria mais difícil.

Agradecimentos

Agradeço a meu pai, minha mãe.

"Nós sempre nos definimos pela capacidade de superar o impossível."
(Joseph Cooper)

Resumo

Aurelio, Giovanni, **Previsão de Emissões de CO₂ de Usinas Termelétricas Nacionais utilizando Redes Neurais Artificiais**. Trabalho de Conclusão de Curso (Graduação em Engenharia Elétrica). Centro de Ciências Exatas e de Tecnologia, Universidade Federal de São Carlos. 59 p. São Carlos, 2026.

Embora a maior parte da energia elétrica no Brasil provenha de usinas hidroelétricas, as termoeletricas são fundamentais para a segurança energética nacional. No entanto, com as mudanças climáticas causadas pelas emissões de CO₂ de diversos setores econômicos, é crucial equilibrar essa segurança com as emissões de CO₂ no setor elétrico brasileiro. Este trabalho propõe estimar as emissões horárias de CO₂ equivalente (CO₂-e) das termoeletricas brasileiras usando redes neurais para apoiar o planejamento energético e ambiental do Sistema Interligado Nacional. Enquanto a literatura baseia a modelagem dessas emissões em funções específicas para cada usina, as emissões no Brasil são inventariadas pela Eletrobras e pelo Instituto de Energia e Meio Ambiente em uma base anual. Para contornar isso, este estudo discretiza as emissões anuais de cada usina em estimativas horárias, utilizando regressão não linear em função de dados disponibilizados pelo Operador Nacional do Sistema Elétrico. Com isso, é possível generalizar as estimativas de emissões horárias das termoeletricas brasileiras. Este trabalho inclui análises para quatro termoeletricas com diferentes capacidades de geração, ciclos de operação e combustíveis, fornecendo *insights* para o planejamento energético, elétrico e ambiental do país.

Palavras-Chave: palavrachave1; palavrachave2; palavrachave3.

Abstract

Aurelio, Giovanni, **CO₂ Emissions Forecasting for National Thermoelectric Power Plants using Artificial Neural Networks**. Final year thesis (Bachelor degree in Electrical Engineering). Exact and Technology Sciences Center, Federal University of São Carlos. 59 p. São Carlos, 2026.

Although most of Brazil's electric energy comes from hydroelectric plants, thermal power plants are crucial for national energy security. However, with climate change caused by CO₂ emissions from various economic sectors, it is crucial to balance this security with CO₂ emissions in the Brazilian electricity sector. This work proposes estimating hourly CO₂ equivalent (CO₂-e) emissions from Brazilian thermal power plants using neural networks to support energy and environmental planning of the National Interconnected System. While the literature bases the modeling of these emissions on specific functions for each plant, emissions in Brazil are inventoried by Eletrobras and the Instituto de Energia e Meio Ambiente on an annual basis. To overcome this, this work discretizes the annual emissions of each plant into hourly estimates using nonlinear regression based on data provided by the Operador Nacional do Sistema Elétrico. This enables generalizing the estimates of hourly emissions from Brazilian thermal power plants. This work includes analyses for four thermal power plants with different generation capacities, operation cycles, and fuels, providing insights for energy, electric, and environmental planning in the country.

Keywords: keyword1; keyword2; keyword3.

Lista de Figuras

Figura 1 – Quantidade de Usinas por Erro MAE	29
Figura 2 – Distribuição das Usinas por Erro MAE	30
Figura 3 – Comparação de Desempenho Favorável e Não Favorável	31
Figura 4 – Quantidade de Usinas por Razão de Instabilidade	31
Figura 5 – Relação entre MAE e Instabilidade	32
Figura 6 – Relação entre MAE e MSE	32

Lista de Tabelas

Tabela 1 – Arquitetura da Rede Neural <i>Multilayer Perceptron</i> (MLP)	20
Tabela 2 – Tratamento das Variáveis Estáticas	25
Tabela 3 – Tratamento das Variáveis Dinâmicas	25
Tabela 4 – Divisão Temporal dos Dados	26
Tabela 5 – Hiperparâmetros de Treinamento	27
Tabela 6 – Dados para Minimização Padrão	28
Tabela 7 – Dados para Minimização com Regularização L2	29
Tabela 8 – Resumo Estatístico - Sem Regularização	33
Tabela 9 – Resumo Estatístico - Com Regularização	34

Lista de Siglas e Abreviaturas

CO₂	Dióxido de Carbono
CO₂e	Dióxido de Carbono Equivalente
tCO₂e	Toneladas de Dióxido de Carbono Equivalente
ONS	Operador Nacional do Sistema Elétrico
IEMA	Instituto de Energia e Meio Ambiente
SIN	Sistema Interligado Nacional
CEG	Cadastro Específico de Geração
AMPL	<i>A Mathematical Programming Language</i>
IPOPT	<i>Interior Point Optimizer</i>
MLP	<i>Multilayer Perceptron</i>
GRU	<i>Gated Recurrent Units</i>
LSTM	<i>Long Short-Term Memory</i>
ReLU	<i>Rectified Linear Unit</i>
MSE	<i>Mean Squared Error</i>
MAE	<i>Mean Absolute Error</i>

Sumário

1	INTRODUÇÃO	12
2	REVISÃO DE LITERATURA	14
2.1	Seção XX	14
2.1.1	Subseção XXX	14
2.1.2	Subseção yyy	14
3	METODOLOGIA	15
3.1	Visão Geral	15
3.2	Fontes de Dados e Descrição das Variáveis	15
3.3	Tratamento das Sazonalidades Temporais	16
3.4	Modelagem das Transições Operacionais	17
3.4.1	Definição de Categorias	17
3.4.2	Identificação de Transições	17
3.4.3	Duração e Fase da Transição.	17
3.5	Formulação do Problema de Calibração	18
3.5.1	Calibração Padrão	18
3.5.2	Regularização L2	19
3.6	Emissões Sintéticas como Alvos para Aprendizado	19
3.7	Arquitetura da Rede Neural	19
3.7.1	Estrutura das Camadas	20
3.7.2	Estratégias de Regularização Adicionais	20
3.8	Métricas de Avaliação	20
4	DESENVOLVIMENTO	22
4.1	Organização Geral da Implementação	22
4.2	Implementação do Pré-Processamento	22
4.2.1	Filtragem das Usinas e Organização Temporal	22
4.2.2	Classificação das Categorias de Geração	23
4.2.3	Identificação de Transições	23
4.3	Implementação da Calibração no <i>A Mathematical Programming Language</i> (AMPL)	23
4.3.1	Estrutura do Modelo de Otimização	23
4.3.2	Integração Python-AMPL	23
4.3.3	Ajuste do Parâmetro de Regularização λ	24
4.3.4	Geração das Emissões Sintéticas Horárias	24

4.4	Preparação das Features para a Rede Neural	24
4.4.1	Tratamento das Variáveis	24
4.4.2	Cálculo da Duração e Fase da Transição	25
4.5	Implementação da Rede Neural	25
4.5.1	Construção da Arquitetura MLP	25
4.5.2	Ponderação Diferenciada para Transições	26
4.5.3	Early Stopping e Gradient Clipping	26
4.6	Divisão dos Dados e Configuração do Treinamento	26
5	RESULTADOS	28
5.1	Visão Geral dos Experimentos	28
5.2	Análise do Algoritmo de Minimização	28
5.3	Desempenho da Rede Neural	29
5.3.1	Distribuição Geral dos Erros	29
5.3.2	Melhores e Piores Desempenhos	30
5.3.3	Análise de Estabilidade	31
5.3.4	Relação entre Métricas	32
5.4	Síntese e Discussão	33
6	CONCLUSÕES	35
A	TÍTULO DO APÊNDICE 1	36
A	TÍTULO DO ANEXO 1	59
A.1	Seção do anexo 1	59
A.1.1	Subseção do anexo 1	59

1 Introdução

O setor elétrico brasileiro possui uma matriz predominantemente renovável, com destaque para as usinas hidrelétricas, que historicamente respondem pela maior parte da geração de energia elétrica no país. No entanto, períodos de estiagem prolongada, aliados ao crescimento da demanda e à necessidade de segurança energética, exigem o acionamento frequente de usinas termelétricas. Embora essas usinas sejam fundamentais para garantir o suprimento contínuo de energia, sua operação está associada à emissão de gases de efeito estufa, especialmente o Dióxido de Carbono (CO_2). As mudanças climáticas, impulsionadas pelas emissões de diversos setores econômicos, têm imposto desafios crescentes aos planejadores energéticos. No Brasil, o Sistema Interligado Nacional (SIN) requer ferramentas que permitam equilibrar a segurança do suprimento elétrico com as metas de descarbonização assumidas pelo país em acordos internacionais. Nesse contexto, a estimação precisa das emissões de CO_2 das termelétricas em escala horária torna-se um insumo relevante para o despacho econômico-ambiental e para o planejamento da operação.

A literatura internacional apresenta modelos consolidados para a estimação de emissões horárias de usinas termelétricas, geralmente baseados em funções quadráticas ou exponenciais calibradas a partir de medições contínuas ou de dados de projeto de cada usina. No entanto, o cenário brasileiro apresenta uma particularidade: os dados de emissões de CO_2 das termelétricas são inventariados anualmente — e não horariamente — por instituições como o Instituto de Energia e Meio Ambiente (IEMA) e, anteriormente, pela Eletrobras. Essa disponibilidade exclusivamente anual de dados impõe um problema de natureza inversa: é necessário estimar o comportamento horário das emissões a partir de agregados anuais. Além disso, as termelétricas brasileiras operam com diferentes tipos de combustível (gás natural, óleo diesel, carvão mineral, biomassa), diferentes ciclos de operação (ciclo simples, ciclo combinado) e operam em patamares discretos de carga, com transições entre níveis de geração que afetam a eficiência e, conseqüentemente, as emissões. A literatura nacional ainda carece de metodologias sistemáticas para a discretização temporal das emissões anuais em estimativas horárias que considerem essas particularidades operacionais e que sejam generalizáveis para diferentes usinas.

Diante desse cenário, este trabalho propõe uma metodologia para estimar as emissões horárias de Dióxido de Carbono Equivalente (CO_2e) das usinas termelétricas brasileiras, combinando técnicas de otimização não-linear e redes neurais artificiais. A metodologia estrutura-se em duas etapas principais. Na primeira etapa, calibram-se os coeficientes de uma equação paramétrica de emissão (composta por termos quadrático e exponencial) por meio de um problema de minimização resolvido em AMPL com o solver *Interior Point Optimizer* (IPOPT), utilizando como referência os totais anuais do IEMA. Dois cenários

são comparados: calibração padrão e calibração com regularização L2. Na segunda etapa, as emissões sintéticas horárias geradas tornam-se alvos para o treinamento de uma rede neural MLP, que aprende diretamente a relação entre variáveis operacionais (geração, categoria de operação, fase de transição), características estáticas da usina (combustível, potência instalada, eficiência) e as emissões horárias. Mais especificamente, o trabalho desenvolve um algoritmo de classificação de categorias de geração e identificação de transições operacionais com base em limiares adaptativos, calibra os coeficientes da equação de emissão para 39 usinas termelétricas brasileiras no período de 2020 a 2024, treina e valida uma rede neural MLP para estimar emissões horárias, e compara o desempenho dos cenários de calibração (padrão vs. L2) em termos de precisão e estabilidade.

Como contribuições principais, este trabalho oferece uma metodologia sistemática para discretização de emissões anuais em estimativas horárias aplicável a termelétricas com diferentes combustíveis e ciclos de operação; uma base de dados integrada combinando informações da Operador Nacional do Sistema Elétrico (ONS) e do IEMA para 39 usinas em um período de 5 anos, com enriquecimento de *features* temporais (categorias, transições, fases); uma comparação quantitativa entre calibração padrão e com regularização L2, fornecendo subsídios para a escolha da estratégia mais adequada em problemas de calibração inversa no setor elétrico; e um modelo de rede neural treinado para estimação horária de emissões, com análise de desempenho por categoria operacional e tipo de combustível. Essas contribuições podem apoiar o planejamento energético e ambiental do Sistema Interligado Nacional (SIN), fornecendo insumos horários para modelos de despacho econômico-ambiental e para a avaliação de políticas de descarbonização.

2 Revisão de Literatura

2.1 Seção XX

XXX

2.1.1 Subseção XXX

XXXX

2.1.2 Subseção yyy

yyyy

3 Metodologia

3.1 Visão Geral

A estimação horária de emissões de CO₂ em usinas termelétricas configura-se como um problema inverso: dispõe-se de medições anuais agregadas, fornecidas pelos inventários do IEMA, mas necessita-se do comportamento horário para aplicações em planejamento energético e despacho econômico-ambiental. Além disso, a relação entre geração horária e emissão é intrinsecamente não-linear (**INSERIR REFERÊNCIA DO ARTIGO**).

Para enfrentar esses desafios, a metodologia proposta estrutura-se em duas etapas complementares. Na primeira etapa, resolve-se um problema de otimização não-linear para calibrar os coeficientes de uma equação paramétrica de emissão horária, utilizando como referência os totais anuais disponibilizados pelo IEMA. Para mitigar o mal condicionamento desse problema inverso, aplica-se regularização L2, sendo comparados dois cenários: calibração padrão e calibração com regularização.

Na segunda etapa, as emissões sintéticas horárias geradas pela equação calibrada tornam-se o alvo para o treinamento de uma rede neural MLP. A rede aprende diretamente a relação entre as variáveis operacionais (geração, categoria de operação, fase de transição), as características estáticas da usina (combustível, potência instalada, eficiência) e a emissão horária, sem incorporar explicitamente a equação física na função de perda.

Essa abordagem em cascata permite:

- Garantir consistência com os totais anuais do IEMA;
- Capturar não-linearidades que a equação paramétrica pode não expressar plenamente;
- Avaliar, por meio da comparação entre os cenários padrão e L2, qual estratégia de calibração produz alvos sintéticos mais coerentes.

3.2 Fontes de Dados e Descrição das Variáveis

Foram empregadas duas fontes principais de dados, abrangendo o período de 2020 a 2024.

Operador Nacional do Sistema Elétrico (ONS)

Por meio da iniciativa Dados Abertos, obtém-se a geração horária de todas as usinas do Sistema Interligado Nacional (SIN). Os registros incluem: identificação da usina, data,

hora e geração horária em MW. O conjunto é filtrado para reter apenas usinas termelétricas.

Instituto de Energia e Meio Ambiente (IEMA)

Os inventários anuais do IEMA fornecem:

- Emissões totais anuais de Toneladas de Dióxido de Carbono Equivalente (tCO₂e) por usina;
- Características cadastrais fixas: potência instalada (MW), fator de capacidade (%), eficiência energética (%), tipo de combustível e ciclo de operação (ciclo combinado, ciclo simples, cogeração).

Usinas de cogeração são excluídas da análise por não apresentarem, nos inventários do IEMA, informações completas de potência instalada e eficiência. Após a integração das fontes e a remoção de registros faltantes, restam 39 usinas termelétricas com dados completos para o período 2020-2024. As emissões anuais são mantidas em tCO₂e, e a geração horária em MW.

3.3 Tratamento das Sazonalidades Temporais

A variável temporal “Índice”, que representam a posição da hora dentro do ano, não pode ser tratada como grandeza ordinal linear, pois a relação entre instantes próximos ao final e ao início de um ano ou período não é capturada por uma codificação escalar convencional.

Para preservar a periodicidade natural do calendário, adota-se uma codificação polar. Ou seja, dado um índice $i \in [1, H]$, onde $H \in \{8760, 8784\}$, representando as horas do respectivo ano, definem-se as seguintes transformações:

$$\text{Seno_Indice}(i) = \sin\left(\frac{2\pi i}{H}\right),$$

$$\text{Cosseno_Indice}(i) = \cos\left(\frac{2\pi i}{H}\right)$$

Dessa forma, ao agrupar os dados de forma crescente em relação a variável “Ano”, essa representação mapeia valores temporais para pontos sobre o círculo unitário, de modo que instantes próximos tornam-se vizinhos no espaço polar, refletindo sua continuidade temporal.

3.4 Modelagem das Transições Operacionais

Termelétricas operam tipicamente em patamares discretos de carga, não em regime continuamente variável. Para capturar esse comportamento, define-se um sistema de categorias de geração com base na proximidade relativa entre valores de potência.

3.4.1 Definição de Categorias

Dada a série de geração horária $G = [g_1, g_2, \dots, g_T]$ em MW, valores inferiores a 20 MW são classificados na categoria 1, pois é considerado uma carga muito baixa ou desligamento. Para os demais valores, dois pontos pertencem à mesma categoria se sua diferença relativa for inferior ou igual a um limiar $\theta = 15\%$ do maior valor do grupo.

Formalmente, g_i e g_j pertencem à mesma categoria se:

$$\frac{|g_i - g_j|}{\max(g_i, g_j)} \leq \theta$$

O limiar de 15% foi definido com base na análise exploratória da distribuição de carga nos dados da ONS, representando o valor típico que separa regimes operacionais distintos.

3.4.2 Identificação de Transições

Complementando a definição das categorias de geração, foi implementado um algoritmo para identificação das gerações em estágio de transição entre categorias.

Isto é, seja c_t a categoria no instante t . Define-se a moda m_t das categorias em uma janela de três horas anteriores e três horas posteriores, excluindo o próprio instante:

$$m_t = \text{moda}\{c_{t-3}, c_{t-2}, c_{t-1}, c_{t+1}, c_{t+2}, c_{t+3}\}$$

O instante t é classificado como transição se $c_t \neq m_t$. Com isso, é atribuído um valor de categoria 0 para esses dados de geração. Este critério captura períodos de rampa de carga como partidas, paradas e variações controladas, nos quais o fator de emissão horária pode diferenciar do regime estabilizado.

3.4.3 Duração e Fase da Transição.

Para cada evento de transição contínuo, ou seja, sequência de horas consecutivas classificadas como transição, definem-se:

- Duração: Número de horas consecutivas do evento;
- Fase: Posição relativa dentro do evento, classificada como 1 (início), 2 (meio) ou 3 (fim).

Essas variáveis permitem que o modelo compreenda a transição como um processo contínuo, com dinâmica de emissão potencialmente distinta ao longo de suas fases.

3.5 Formulação do Problema de Calibração

A relação entre geração horária e emissão horária é modelada por uma equação com termo quadrático e termo exponencial, conforme proposta na literatura (**INSERIR REFERÊNCIA**):

$$E_k^{\text{horária}}(P_{G_i}) = (\alpha_k P_{G_i}^2 + \beta_k P_{G_i} + \gamma_k) + \omega_k e^{\mu_k P_{G_i}} \quad (3.1)$$

onde:

- P_{G_i} representa a geração em MW, dividida por 100 para normalização dos dados.
- $\alpha, \beta, \gamma, \omega, \mu$ são coeficientes a serem calibrados por usina.

3.5.1 Calibração Padrão

Os coeficientes são obtidos resolvendo o seguinte problema de minimização, que busca minimizar o erro quadrático médio entre a emissão anual estimada e a emissão anual real fornecida pelo IEMA:

$$\begin{aligned} \min \quad & \frac{1}{N} \cdot \left[\sum_{j=1}^N \left(\sum_{i=1}^h (E_{usina}^i(P_{G_i})) - E_{usina}^j \right)^2 \right] \\ \text{s. a:} \quad & \alpha_k \geq 0 \\ & \beta_k \\ & \gamma_k \\ & \omega_k \\ & \mu_k \end{aligned} \quad (3.2)$$

onde

$$\left\{ \begin{array}{l} N : \text{Número de usinas} \\ H : \text{Horas do respectivo ano} \\ E_{usina}^j : \text{Emissão anual apresentada pelo IEMA para usina } j \end{array} \right.$$

A restrição de não-negatividade sobre α garante que a contribuição dos respectivos termos seja fisicamente coerente.

3.5.2 Regularização L2

O problema de calibração inversa é intrinsecamente mal condicionado, podendo produzir coeficientes excessivamente elevados que se cancelam mutuamente através de overfitting.

Para mitigar esse efeito, introduz-se um termo de regularização L2 à função objetivo:

$$\begin{aligned}
 \min \quad & \frac{1}{N} \cdot \left[\sum_{j=1}^N \left(\sum_{i=1}^h \left(E_{usina}^i(P_{G_i}) \right) - E_{usina}^j \right)^2 \right] + \lambda \cdot (\alpha^2 + \beta^2 + \gamma^2 + \omega^2 + \mu^2) \\
 \text{s. a:} \quad & \alpha_k \geq 0 \\
 & \beta_k \\
 & \gamma_k \\
 & \omega_k \\
 & \mu_k \\
 & \lambda
 \end{aligned} \tag{3.3}$$

O parâmetro $\lambda \geq 0$ controla a intensidade da regularização, ou seja, penaliza coeficientes de grande magnitude, promovendo soluções mais estáveis e generalizáveis. O valor de λ deve ser selecionado de forma a equilibrar a redução do erro de calibração e a estabilidade dos coeficientes.

3.6 Emissões Sintéticas como Alvos para Aprendizado

Uma vez calibrados os coeficientes para os cenários padrão e com regularização L2, aplica-se a Equação 3.1 sobre a série horária de geração para produzir as emissões sintéticas horárias $\hat{E}_t$. Essas emissões sintéticas tornam-se a variável alvo para o treinamento supervisionado da rede neural.

É importante observar que a rede neural, ao ser treinada com esses alvos sintéticos, herda os erros de modelagem da equação paramétrica e da calibração. Portanto, a comparação entre os cenários padrão e L2 não avalia apenas a qualidade da rede, mas também qual estratégia de calibração produz alvos mais consistentes para o aprendizado supervisionado.

3.7 Arquitetura da Rede Neural

Optou-se por uma arquitetura do tipo MLP em detrimento de redes recorrentes, como *Long Short-Term Memory* (LSTM) ou *Gated Recurrent Units* (GRU), porque a emissão

horária, no modelo proposto, depende essencialmente da geração no mesmo instante t , e não de uma sequência temporal longa. A MLP é suficiente, mais simples e menos suscetível a overfitting com o volume de dados disponível.

3.7.1 Estrutura das Camadas

A Tabela 1 apresenta a arquitetura adotada.

Camada	Neurônios	Ativação	Regularização
Entrada	Variável	-	-
Oculto 1	128	ReLU	Dropout (0,3)
Oculto 2	64	ReLU	Dropout (0,3)
Oculto 3	32	ReLU	Dropout (0,3)
Saída	1	ReLU	-

Tabela 1 – Arquitetura da Rede Neural MLP

A arquitetura em funil reduz progressivamente a capacidade representacional, atuando como uma forma de regularização e prevenindo overfitting. A função de ativação *Rectified Linear Unit* (ReLU) é escolhida por sua simplicidade computacional e por mitigar o problema do gradiente transitório em comparação com funções sigmoidais. O dropout com taxa de 0,3 desativa aleatoriamente 30% dos neurônios a cada iteração de treinamento, forçando a rede a aprender representações redundantes e robustas.

3.7.2 Estratégias de Regularização Adicionais

Além do *dropout*, empregam-se as seguintes estratégias de regularização:

- Early stopping: o treinamento é interrompido quando a perda no conjunto de validação não apresenta melhora por um número pré-definido de épocas, restaurando os pesos da melhor época. Isso evita overfitting ao interromper o treinamento antes que a rede memorize ruídos.
- Gradient clipping: a norma L2 do vetor de gradientes é limitada a um valor máximo antes da atualização dos pesos. Esta técnica previne a explosão dos gradientes, fenômeno que pode ocorrer quando a função de perda apresenta regiões de alta curvatura ou quando amostras com pesos elevados produzem gradientes desproporcionais.

3.8 Métricas de Avaliação

Para avaliar a qualidade das estimativas, empregam-se as seguintes métricas:

- *Mean Absolute Error* (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Interpretável diretamente na unidade tCO₂e/h.

- *Mean Squared Error* (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Penaliza mais severamente erros grandes, sendo útil para detectar a presença de *outliers*.

- Razão de Estabilidade

$$R = \frac{\text{MSE}}{\text{MAE}^2}$$

A razão R é empregada como indicador da estabilidade do modelo:

$$\left\{ \begin{array}{l} R < 3 : \text{ Modelo estável, erros concentrados em torno da média;} \\ 3 \leq R \leq 5 : \text{ Instabilidade moderada, presença de alguns } \textit{outliers}; \\ R > 5 : \text{ Presença significativa de } \textit{outliers}, \text{ modelo pouco confiável.} \end{array} \right.$$

4 Desenvolvimento

4.1 Organização Geral da Implementação

Todo o código foi desenvolvido na linguagem Python, versão 3.10.12, utilizando as bibliotecas Pandas e NumPy para manipulação dos dados, AMPL (via biblioteca *amplpy*) para a calibração dos coeficientes, e PyTorch para a construção e treinamento da rede neural.

O desenvolvimento foi estruturado em três módulos principais:

- **Módulo 1 – Pré-processamento:** filtragem das usinas, organização temporal, classificação de categorias, identificação de transições e cálculo de duração/fase.
- **Módulo 2 – Calibração (AMPL/IPOPT):** modelagem do problema de otimização, integração Python-AMPL, ajuste do parâmetro de regularização e geração das emissões sintéticas.
- **Módulo 3 – Rede Neural (PyTorch):** construção da arquitetura MLP, ponderação das amostras, treinamento com early stopping e gradient clipping.

Todos os experimentos foram executados em um computador com processador AMD Ryzen 5 5600X (12 núcleos), 16 GB de RAM e GPU NVIDIA RTX 5070 (12 GB), sob sistema operacional Windows 11.

4.2 Implementação do Pré-Processamento

4.2.1 Filtragem das Usinas e Organização Temporal

O primeiro passo consistiu em integrar as duas fontes de dados e garantir a consistência do conjunto para o período analisado (2020 a 2024). A partir dos dados da ONS, filtrou-se inicialmente apenas as usinas termelétricas.

Em seguida, implementou-se um algoritmo para reter apenas as usinas que possuíam registros completos em todos os inventários anuais do IEMA para o período de interesse. Usinas de cogeração foram automaticamente excluídas devido à ausência de informações de potência instalada e eficiência energética nos inventários.

Após a filtragem, cada usina foi identificada unicamente por seu Cadastro Específico de Geração (CEG). Os dados horários de geração de cada usina foram então separados por ano e concatenados em uma única série temporal por usina, preservando duas colunas fundamentais: “Ano” (2020 a 2024) e “Índice” (posição da hora dentro do ano, variando

de 1 a 8760, ou 8784 para anos bissextos). Esta organização permitiu manter a estrutura temporal necessária para etapas subsequentes.

4.2.2 Classificação das Categorias de Geração

O algoritmo de classificação de categorias foi implementado conforme o procedimento descrito na Metodologia. A implementação percorre todos os valores de geração acima de 20 MW, agrupando aqueles cuja diferença relativa em relação à média do grupo não excede o limiar de 15%. Após a formação inicial dos grupos, realiza-se uma etapa iterativa de mesclagem, na qual grupos cujas médias diferem em menos de 15% são combinados. Por fim, valores abaixo do limiar de 20 MW são atribuídos à categoria 1, pois representam carga muito baixa ou desligamento.

4.2.3 Identificação de Transições

Para identificar horas em transição, implementou-se uma função que percorre o vetor de categorias com uma janela deslizante de três horas anteriores e três horas posteriores. Para cada hora central da janela, calcula-se a moda das categorias nos seis instantes vizinhos. Se a categoria da hora central for diferente da moda dos vizinhos, a hora é marcada como transição.

As horas nas bordas do ano utilizam apenas as informações disponíveis para determinação de sua classificação.

4.3 Implementação da Calibração no AMPL

4.3.1 Estrutura do Modelo de Otimização

O problema de minimização foi modelado na linguagem AMPL. Foram criados dois arquivos: um arquivo de modelo (*.mod*) contendo as variáveis, a função objetivo e as restrições; e um arquivo de comandos (*.run*) especificando as opções do solver IPOPT.

A função objetivo implementa a Equação **INSERIR NUMERAÇÃO** para o cenário padrão, com a adição do termo de regularização L2. Os dados de geração horária e a emissão anual alvo são fornecidos separadamente.

4.3.2 Integração Python-AMPL

Para automatizar a calibração das 39 usinas, utilizou-se a biblioteca *amplpy*. O fluxo de integração foi o seguinte:

- Para cada usina, extraem-se os dados de geração horária e a emissão anual correspondente do IEMA;

- Os dados são escritos em um arquivo *.dat* para cada usina;
- O modelo AMPL é iterado em cada usina, com seu respectivo arquivo *.dat* importado;
- O solver IPOPT é executado;
- Os coeficientes otimizados são extraídos e armazenados em um dicionário Python.

Este processo foi executado em paralelo utilizando a biblioteca *multiprocessing*, reduzindo o tempo total de calibração de aproximadamente 8 horas em processamento serial para cerca de 2 horas.

4.3.3 Ajuste do Parâmetro de Regularização λ

O parâmetro λ foi ajustado por meio de testes empíricos com os valores 1, 10, 30 e 100. Para cada valor, calibrou-se um subconjunto de 10 usinas representativas e avaliou-se o erro MAE médio e a variância dos coeficientes entre anos consecutivos.

O valor $\lambda = 30$ foi selecionado por apresentar o melhor equilíbrio. Valores superiores como $\lambda = 100$ introduziram viés excessivo, enquanto valores inferiores como $\lambda = 1$ não foram suficientes para estabilizar os coeficientes.

4.3.4 Geração das Emissões Sintéticas Horárias

Uma vez calibrados os coeficientes para cada usina, aplicou-se a Equação **INSERIR NUMERO** sobre a série horária de geração para produzir as emissões sintéticas. Este procedimento foi executado separadamente para os dois cenários, padrão e com regularização L2.

4.4 Preparação das Features para a Rede Neural

4.4.1 Tratamento das Variáveis

Antes do treinamento, as variáveis de entrada foram preparadas conforme as Tabelas 2 e 3.

Variável	Tratamento	Justificativa
Combustível	One-Hot Encoding	Categoria nominal sem ordem intrínseca
Ciclo de operação	One-Hot Encoding	Categoria nominal
Potência instalada	Sem normalização	Possui significado físico direto
Eficiência energética	Sem normalização	Possui significado físico direto
Fator de capacidade	Sem normalização	Possui significado físico direto

Tabela 2 – Tratamento das Variáveis Estáticas

Variável	Tratamento	Justificativa
Geração (MW)	Standard Scaler	Ajustado após divisão treino/validação/teste
Duração da transição (h)	Standard Scaler	Ajustado para rede neural
Fase da transição	Standard Scaler	Ajustado para rede neural
Índice temporal	Seno/Cosseno (Codificação polar)	Preserva periodicidade anual
Ano	Seno/Cosseno (Codificação polar)	Permite capturar tendências anuais não lineares

Tabela 3 – Tratamento das Variáveis Dinâmicas

4.4.2 Cálculo da Duração e Fase da Transição

Ademais, o cálculo da duração e fase das gerações de transição foi implementado varrendo o vetor binário de transição e identificando blocos consecutivos onde o valor é 1. Para cada bloco, a duração é o comprimento do bloco, e a fase é atribuída conforme a posição relativa: primeira hora = 1, última hora = 3, horas intermediárias = 2. Transições de duração 1 recebem fase = 1.

4.5 Implementação da Rede Neural

4.5.1 Construção da Arquitetura MLP

A rede neural foi implementada utilizando PyTorch. A escolha desta biblioteca deveu-se à possibilidade de executar o treinamento, validação e teste utilizando a GPU da

máquina, acelerando significativamente o processamento.

A arquitetura seguiu a estrutura definida na Tabela **INSERIR NUMERO**: três camadas ocultas com 128, 64 e 32 neurônios, ativação ReLU, *dropout* de 0,3 após cada camada oculta, e camada de saída com um neurônio e ativação ReLU para garantir não-negatividade das emissões. O horizonte de previsão foi definido como 1 hora.

4.5.2 Ponderação Diferenciada para Transições

Para melhorar a capacidade do modelo em aprender eventos de transição, implementou-se um sistema de ponderação na função de perda. Cada amostra recebeu um peso individual: 1,0 para horas em regime estável e 5,0 para horas classificadas como transição.

O valor 5 foi escolhido empiricamente: pesos menores como 2 ou 3 não produziram melhora significativa no erro das transições, enquanto pesos maiores como 10 causaram overfitting a esses eventos.

4.5.3 Early Stopping e Gradient Clipping

O método early stopping foi implementado manualmente no loop de treinamento, visando obter os melhores valores para os pesos. Assim, se nenhuma melhora fosse observada após 50 épocas consecutivas, o treinamento é interrompido e os pesos da melhor época eram restaurados.

O gradient clipping atua juntamente, calculando o gradiente e atualizando os pesos para mantê-los no limiar de $[-1, 1]$, evitando que os gradientes se tornem excessivamente grandes durante a retropropagação.

4.6 Divisão dos Dados e Configuração do Treinamento

A divisão temporal dos dados foi configurada conforme a Tabela 4

Conjunto	Período	Finalidade
Treinamento	2020, 2021, 2022	Ajuste dos pesos
Validação	2023	Early stopping, seleção de hiperparâmetros
Teste	2024	Avaliação final do modelo

Tabela 4 – Divisão Temporal dos Dados

Os hiperparâmetros de treinamento foram configurados conforme a Tabela 5

Métrica	Fórmula	Justificativa
MAE (tCO ₂ /h)	$\frac{1}{n} \sum y_i - \hat{y}_i$	Interpretação direta na unidade original
MSE (tCO ₂ /h) ²	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Penaliza erros grandes
RMSE (tCO ₂ /h)	$\sqrt{MSE}$	Mesma unidade do MAE, sensível a outliers

Tabela 5 – Hiperparâmetros de Treinamento

5 Resultados

5.1 Visão Geral dos Experimentos

Foram realizados experimentos para os dois cenários propostos na metodologia: calibração padrão e calibração com regularização L2. Para cada cenário, calibrou-se a equação de emissão para 39 usinas termelétricas no período de 2020 a 2024, geraram-se as emissões sintéticas horárias e treinou-se uma rede neural MLP com a arquitetura descrita na Seção 3.7. O conjunto de treinamento compreendeu os anos de 2020 a 2022, o conjunto de validação o ano de 2023, e o conjunto de teste o ano de 2024.

Este capítulo apresenta os resultados obtidos, organizados em três blocos principais:

1. Análise da calibração dos coeficientes no AMPL;
2. Desempenho da rede neural nas estimativas horárias;
3. Síntese e discussão dos achados, com ênfase na comparação entre os cenários padrão e L2.

5.2 Análise do Algoritmo de Minimização

As Tabelas 6 e 7 apresentam um resumo estatístico dos coeficientes calibrados para os dois cenários, considerando todas as 39 usinas e os 5 anos analisados.

Coeficiente	Média	Desvio Padrão	Mínimo	Máximo
Alpha	892.79	4655.59	0.0	28974.21
Beta	301.12	4471.49	-19549.63	18262.05
Gamma	18617.37	57958.79	-57831.02	282549.77
Omega	-18531.67	57979.78	-282549.76	57833.15
Mu	-11055.89	69140.89	-431769.14	189.05
MSE	17693.99	42001.18	0.0	231081.89

Tabela 6 – Dados para Minimização Padrão

Coeficiente	Média	Desvio Padrão	Mínimo	Máximo
Alpha	5.87	8.65	0.0	35.05
Beta	8.69	13.37	-15.93	45.92
Gamma	0.21	6.87	-20.44	28.09
Omega	4.79	6.50	-7.65	23.37
Mu	2.96	3.96	-3.23	17.41
MSE	53713.20	93047.72	510.96	432081.27

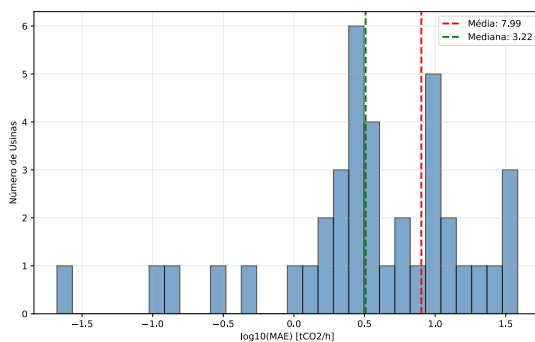
Tabela 7 – Dados para Minimização com Regularização L2

Com isso, ainda que o valor de MSE para a minimização com regularização L2 seja maior, este método promoveu valores fisicamente confiáveis em comparação com os valores obtidos pelo método de minimização padrão.

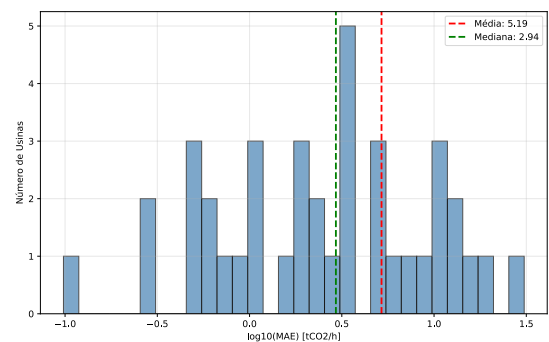
5.3 Desempenho da Rede Neural

5.3.1 Distribuição Geral dos Erros

As Figuras 1a e 1b mostram os histogramas do erro MAE por usina obtidos no conjunto de teste (ano de 2024), respectivamente para o método de minimização padrão e para a abordagem com regularização L2. A escala logarítmica adotada evidencia a distribuição dos erros entre as 39 usinas analisadas.



(a) Sem regularização



(b) Com regularização L2

Figura 1 – Quantidade de Usinas por Erro MAE

A Figura 1a apresenta o histograma do MAE para o cenário de minimização padrão. Observa-se que a maioria das usinas concentra-se na faixa de MAE entre 1,3 e 3,2 tCO₂e/h, com pico em aproximadamente 1,3 tCO₂! (tCO₂!)/h.

No entanto, a distribuição apresenta uma cauda longa para a direita: 9 usinas apresentaram MAE superior a 5 tCO₂!/h, sendo 3 delas com erros superiores a 19 tCO₂e/h. Esse comportamento indica que, embora o modelo padrão seja capaz de estimar adequadamente as emissões da maioria das usinas, ele falha significativamente para um subconjunto delas.

ESCREVER SOBRE O GRAFICO L2 Em comparação, o cenário com regularização L2 apresentado na Figura 1b apresentou uma distribuição mais concentrada, ou seja, com menor dispersão, reduzindo a cauda direita. **COMPLETAR TEXTO**

As Figuras 2a e 2b complementam a análise dos histogramas ao apresentar o MAE individual de cada usina, ordenado decrescentemente.

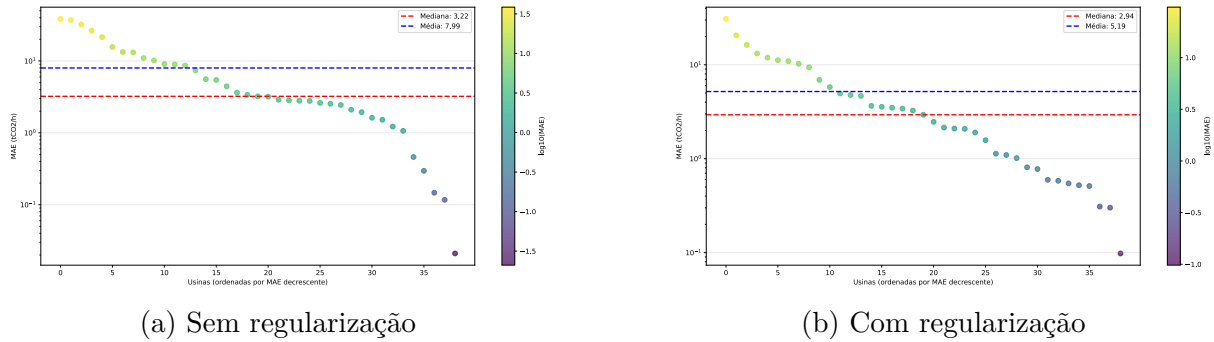


Figura 2 – Distribuição das Usinas por Erro MAE

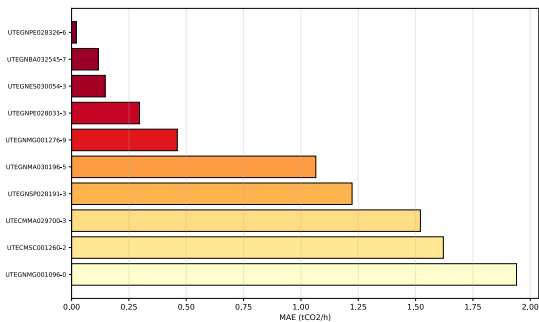
Analisando os dados apresentados, houve uma significativa discrepância entre a mediana e a média para o modelo de minimização padrão, evidenciando a presença de outliers que degradam o desempenho médio do modelo.

Ademais, apenas 5 usinas apresentam MAE superior a 20 tCO₂!/h, sendo responsáveis por puxar a média para um valor muito acima da mediana. Este padrão de distribuição assimétrica confirma que o cenário de minimização padrão, embora adequado para a maioria das usinas, falha significativamente para um subconjunto específico delas.

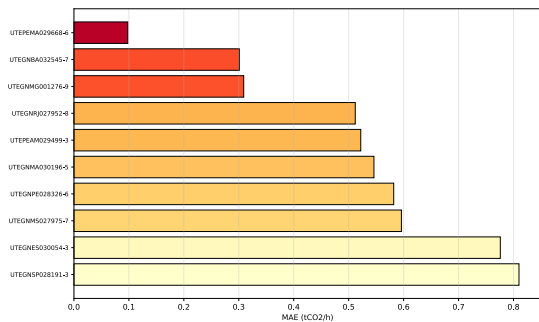
Comparativamente, os dados para o modelo com regularização L2 **COMPLETAR TEXTO REFAZENDO OS GRAFICOS L2**

5.3.2 Melhores e Piores Desempenhos

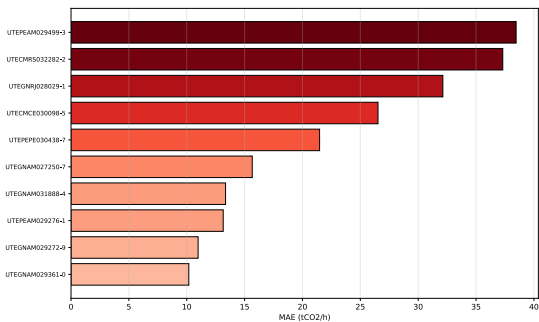
As Figuras 3a e 3c representam os melhores e piores desempenhos das usinas para o método padrão, enquanto que as Figuras 3b e 3d representam os dados para o método com regularização L2.



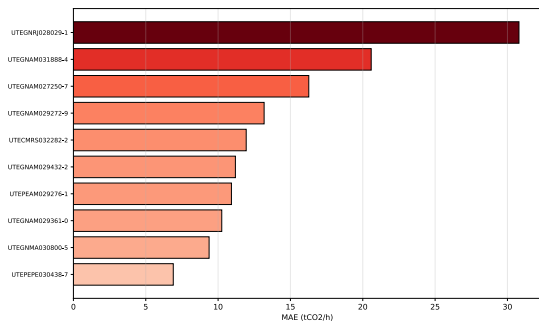
(a) Melhores - Sem regularização



(b) Melhores - Com regularização



(c) Piores - Sem regularização



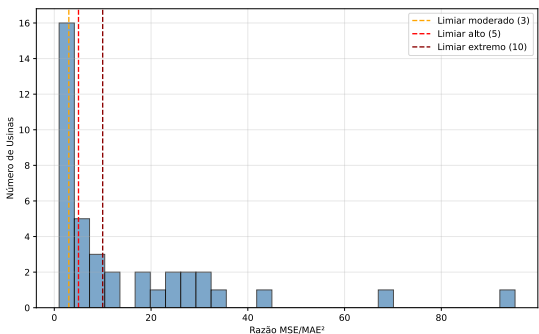
(d) Piores - Com regularização

Figura 3 – Comparação de Desempenho Favorável e Não Favorável

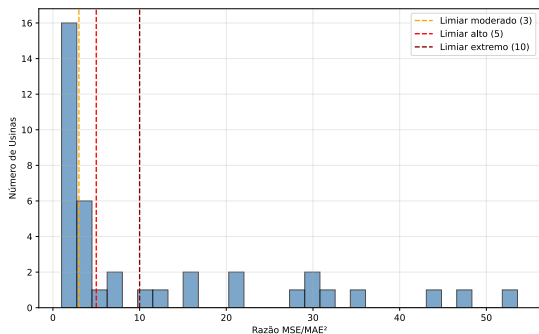
INSERIR TEXTO DE INTERPRETAÇÃO

5.3.3 Análise de Estabilidade

6a 6b



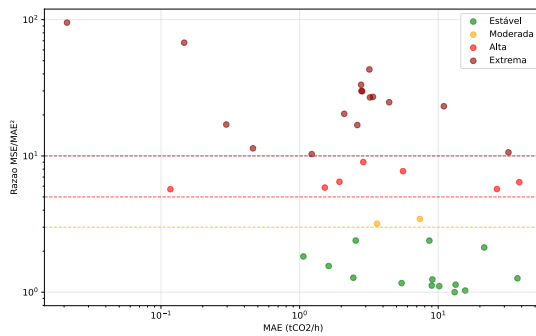
(a) Sem regularização



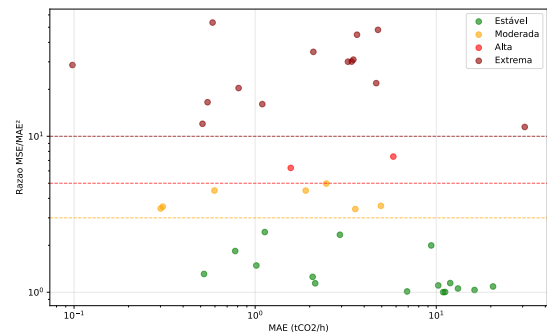
(b) Com regularização

Figura 4 – Quantidade de Usinas por Razão de Instabilidade

5.3.4 Relação entre Métricas

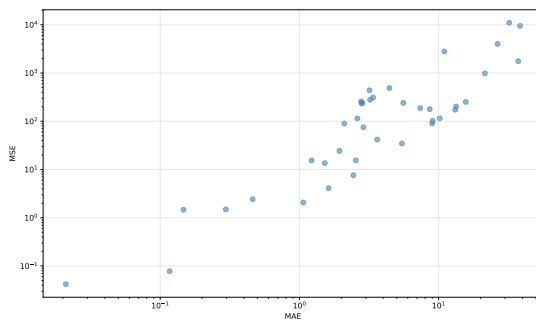


(a) Sem regularização

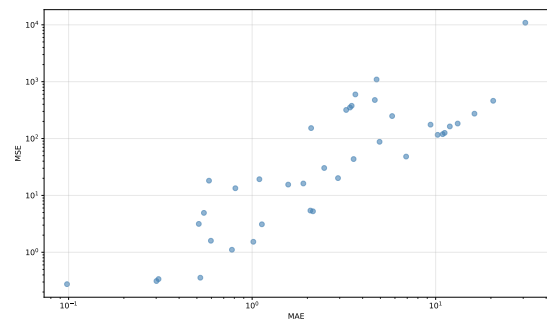


(b) Com regularização

Figura 5 – Relação entre MAE e Instabilidade



(a) Sem regularização



(b) Com regularização

Figura 6 – Relação entre MAE e MSE

5.4 Síntese e Discussão

Métrica	Valor	% do Total
TOTAL DE USINAS	39	-
MAE tCO₂e/h		
Média	7,99	-
Mediana	3,22	-
Desvio Padrão	10,11	-
Mínimo	0,021	-
Máximo	38,46	-
DESEMPENHO		
MAE <10	29	74,4%
MAE <100	39	100,0%
PROBLEMÁTICAS		
MAE >1000	0	0,0%
MAE >10000	0	0,0%

Tabela 8 – Resumo Estatístico - Sem Regularização

Métrica	Valor	% do Total
TOTAL DE USINAS	39	-
MAE tCO₂e/h		
Média	5,19	-
Mediana	2,94	-
Desvio Padrão	6,47	-
Mínimo	0,098	-
Máximo	30,8	-
DESEMPENHO		
MAE <10	31	79,5%
MAE <100	39	100,0%
PROBLEMÁTICAS		
MAE >1000	0	0,0%
MAE >10000	0	0,0%

Tabela 9 – Resumo Estatístico - Com Regularização

6 Conclusões

(??)

A Título do Apêndice 1

Pseudocódigo 1 – Categorização das Gerações

```

ALGORITMO agrupar_geracao
ENTRADA: geracao (lista de valores), limiar (float, opcional = 15)
SAÍDA: grupos (lista de grupos formados), categorias_iniciais (lista de
↪ IDs)

INÍCIO
    categorias <- lista vazia
    grupos <- lista vazia
    proximo_id <- 1

    // Primeira passagem: agrupamento inicial
    PARA i DE 0 ATÉ tamanho(geracao) - 1:
        valor <- geracao[i]

        SE valor <= 20 ENTÃO
            valor <- 0

        SE i == 0 ENTÃO
            lim_inf <- valor * (1 - limiar/100)
            lim_sup <- valor * (1 + limiar/100)

            grupo <- {
                id: proximo_id,
                media: valor,
                limite_inferior: lim_inf,
                limite_superior: lim_sup,
                valores: [valor]
            }

            adicionar grupo em grupos
            adicionar proximo_id em categorias
            proximo_id <- proximo_id + 1

    SENÃO

```

```
grupos_validos <- lista vazia

PARA CADA grupo EM grupos:
  SE grupo.limite_inferior <= valor <=
    ⇨ grupo.limite_superior ENTÃO
    SE grupo.media != 0 ENTÃO
      distancia <- |valor - grupo.media| / grupo.media
      ⇨ * 100
    SENÃO
      distancia <- |valor| * 100

  adicionar (grupo, distancia) em grupos_validos

SE grupos_validos NÃO está vazio ENTÃO
  ordenar grupos_validos por distancia (crescente)
  grupo_escolhido <- grupos_validos[0].grupo

  adicionar valor em grupo_escolhido.valores
  nova_media <- soma(grupo_escolhido.valores) /
    ⇨ tamanho(grupo_escolhido.valores)
  grupo_escolhido.media <- nova_media
  grupo_escolhido.limite_inferior <- nova_media * (1 -
    ⇨ limiar/100)
  grupo_escolhido.limite_superior <- nova_media * (1 +
    ⇨ limiar/100)

  adicionar grupo_escolhido.id em categorias

SENÃO

  lim_inf <- valor * (1 - limiar/100)
  lim_sup <- valor * (1 + limiar/100)

  grupo <- {
    id: proximo_id,
    media: valor,
    limite_inferior: lim_inf,
    limite_superior: lim_sup,
    valores: [valor]
  }
```

```
        adicionar grupo em grupos
        adicionar proximo_id em categorias
        proximo_id <- proximo_id + 1
    FIM SE
FIM SE
FIM PARA

// Segunda fase: mesclar grupos sobrepostos
REPETIR
    houve_mesclagem <- FALSO
    grupos_ordenados <- ordenar grupos por media (crescente)
    novos_grupos <- lista vazia

    i <- 0
    ENQUANTO i < tamanho(grupos_ordenados):
        grupo_atual <- copia(grupos_ordenados[i])
        j <- i + 1

        ENQUANTO j < tamanho(grupos_ordenados):
            proximo_grupo <- grupos_ordenados[j]

            SE grupo_atual.limite_inferior <=
                ↪ proximo_grupo.limite_superior E
                proximo_grupo.limite_inferior <=
                ↪ grupo_atual.limite_superior ENTÃO

                adicionar todos valores de proximo_grupo em
                ↪ grupo_atual.valores
                nova_media <- soma(grupo_atual.valores) /
                ↪ tamanho(grupo_atual.valores)
                grupo_atual.media <- nova_media
                grupo_atual.limite_inferior <- nova_media * (1 -
                ↪ limiar/100)
                grupo_atual.limite_superior <- nova_media * (1 +
                ↪ limiar/100)
                grupo_atual.id <- minimo(grupo_atual.id,
                ↪ proximo_grupo.id)
```

```

        houve_mesclagem <- VERDADEIRO
        j <- j + 1
    SENÃO
        INTERROMPER loop
    FIM SE

    adicionar grupo_atual em novos_grupos
    i <- j

    grupos <- novos_grupos
    ATÉ houve_mesclagem = FALSO

    // Renomear IDs sequencialmente
    grupos_ordenados <- ordenar grupos por media (crescente)
    PARA idx DE 1 ATÉ tamanho(grupos_ordenados):
        grupos_ordenados[idx-1].id <- idx

    RETORNAR grupos_ordenados, categorias

FIM ALGORITMO

```

Pseudocódigo 2 – Identificação de Transições

```

    ALGORITMO identificar_transicoes_e_exportar
    ENTRADA: usina (string), geracao (lista de valores), grupos (lista de
    ↪ grupos),
        categorias_iniciais (lista de IDs), limiar (float, opcional =
    ↪ 15)
    SAÍDA: categorias_finais (lista de inteiros)

    INÍCIO
        n <- tamanho(geracao)

        // Reclassificar todos os valores baseado nos grupos finais
        novas_categorias <- lista vazia

        PARA CADA valor_original EM geracao:
            valor <- SE valor_original <= 20 ENTÃO 0 SENÃO valor_original

```



```
grupo_encontrado <- NULO
PARA CADA grupo EM grupos:
  SE grupo.limite_inferior <= valor <= grupo.limite_superior
    ↪ ENTÃO
      grupo_encontrado <- grupo
    INTERROMPER loop

SE grupo_encontrado != NULO ENTÃO
  novo_id <- grupo_encontrado.id
SENÃO
  grupo_mais_proximo <- grupo em grupos com mínimo
    ↪ |grupo.media - valor|
  novo_id <- grupo_mais_proximo.id

adicionar novo_id em novas_categorias

// Identificar pontos de transição usando janela deslizante
categorias_finais <- copia(novas_categorias)

PARA i DE 0 ATÉ n-1:
  SE categorias_finais[i] == 0 ENTÃO
    CONTINUAR

  // Coletar até 3 categorias anteriores
  categorias_anteriores <- lista vazia
  PARA j DE max(0, i-3) ATÉ i-1:
    adicionar novas_categorias[j] em categorias_anteriores

  // Coletar até 3 categorias posteriores
  categorias_posteriores <- lista vazia
  PARA j DE i+1 ATÉ min(n-1, i+3):
    adicionar novas_categorias[j] em categorias_posteriores

SE tamanho(categorias_anteriores) == 0 OU
  ↪ tamanho(categorias_posteriores) == 0 ENTÃO
  CONTINUAR

cat_atual <- novas_categorias[i]
```

```
// Contar categorias diferentes
diferentes_anteriores <- 0
PARA CADA cat EM categorias_anteriores:
  SE cat != cat_atual ENTÃO
    diferentes_anteriores <- diferentes_anteriores + 1

diferentes_posteriores <- 0
PARA CADA cat EM categorias_posteriores:
  SE cat != cat_atual ENTÃO
    diferentes_posteriores <- diferentes_posteriores + 1

// Verificar se é ponto de transição
SE diferentes_anteriores > tamanho(categorias_anteriores)/2 E
diferentes_posteriores > tamanho(categorias_posteriores)/2
↪ ENTÃO

valor_atual <- SE geracao[i] <= 20 ENTÃO 0 SENÃO geracao[i]

// Calcular média dos valores anteriores
valores_anteriores <- lista vazia
PARA j DE max(0, i-3) ATÉ i-1:
  v <- geracao[j]
  SE v > 20 OU v == 0 ENTÃO
    adicionar v em valores_anteriores

SE valores_anteriores NÃO vazio ENTÃO
  media_anterior <- soma(valores_anteriores) /
  ↪ tamanho(valores_anteriores)
SENÃO
  media_anterior <- valor_atual

// Calcular média dos valores posteriores
valores_posteriores <- lista vazia
PARA j DE i+1 ATÉ min(n-1, i+3):
  v <- geracao[j]
  SE v > 20 OU v == 0 ENTÃO
    adicionar v em valores_posteriores

SE valores_posteriores NÃO vazio ENTÃO
```

```
media_posterior <- soma(valores_posteriores) /  
  ↳ tamanho(valores_posteriores)  
SENÃO  
  media_posterior <- valor_atual  
  
// Calcular diferenças percentuais  
diff_anterior <- |valor_atual - media_anterior| /  
  ↳ max(|media_anterior|, 1) * 100  
diff_posterior <- |valor_atual - media_posterior| /  
  ↳ max(|media_posterior|, 1) * 100  
  
SE diff_anterior > limiar E diff_posterior > limiar ENTÃO  
  categorias_finais[i] <- 0  
  
// Criar diretório de saída  
dir <- caminho_join('Usinas', usina, 'Infos de Geração')  
SE diretorio NÃO existe ENTÃO  
  criar_diretorio(dir)  
  
// Gerar arquivo com informações dos grupos  
dados_grupos <- lista vazia  
PARA CADA grupo EM grupos (ordenados por media crescente):  
  qtd <- 0  
  PARA i DE 0 ATÉ n-1:  
    SE categorias_finais[i] == grupo.id ENTÃO  
      qtd <- qtd + 1  
  
  adicionar em dados_grupos: {  
    'Grupo': grupo.id,  
    'Média': arredondar(grupo.media, 2),  
    'Limite Inferior': arredondar(grupo.limite_inferior, 2),  
    'Limite Superior': arredondar(grupo.limite_superior, 2),  
    'Registros': qtd  
  }  
  
salvar_parquet(dados_grupos, caminho_join(dir, 'Grupos de  
  ↳ Geração.parquet'))  
  
// Gerar arquivo com os pontos de transição
```

```

dados_transicao <- lista vazia
PARA i DE 0 ATÉ n-1:
    SE categorias_finais[i] == 0 ENTÃO
        adicionar em dados_transicao: {
            'Índice': i,
            'Geração': arredondar(geracao[i], 3)
        }

salvar_parquet(dados_transicao, caminho_join(dir, 'Transições de
↪ Geração.parquet'))

RETORNAR categorias_finais

FIM ALGORITMO

```

Pseudocódigo 3 – Modelo AMPL

```

ALGORITMO modelo_regressao_emissoes

// =====
// DECLARAÇÃO DE CONJUNTOS
// =====

CONJUNTO ANO
CONJUNTO HORARIO[a] para cada a em ANO, ordenado: 0 até H[a]

// =====
// DECLARAÇÃO DE PARÂMETROS
// =====

PARÂMETRO H[ANO] // Número de horas por ano
PARÂMETRO emissoes[ANO] // Emissões anuais em kg CO2e
PARÂMETRO pg[ANO, HORARIO] // Geração por ano e hora

PARÂMETRO S_base <- 100 // Base para escala (não utilizado
↪ explicitamente)
PARÂMETRO lambda <- 30 // Parâmetro de regularização L2

// =====

```

```

// DECLARAÇÃO DE VARIÁVEIS
// =====

// Variáveis para regressão padrão
VAR Alpha >= 0 // Coeficiente quadrático (cargas
↪ muito altas)
VAR Beta // Coeficiente linear (eficiência em
↪ carga parcial)
VAR Gamma // Termo constante
VAR Omega // Amplitude do termo exponencial
VAR Mu // Taxa de crescimento exponencial

// Variáveis para regressão com regularização L2
VAR Alpha_L2 >= 0
VAR Beta_L2
VAR Gamma_L2
VAR Omega_L2
VAR Mu_L2

// Variáveis derivadas (emissões horárias estimadas)
VAR Emissao_horaria[ANO, HORARIO] =
    Alpha * pg[a,h]^2
    + Beta * pg[a,h]
    + Gamma
    + Omega * exp(Mu * pg[a,h])

VAR Emissao_horaria_L2[ANO, HORARIO] =
    Alpha_L2 * pg[a,h]^2
    + Beta_L2 * pg[a,h]
    + Gamma_L2
    + Omega_L2 * exp(Mu_L2 * pg[a,h])

// =====
// RESTRIÇÕES FÍSICAS
// =====

// Restrição 1: Emissões horárias nunca negativas
PARA CADA a em ANO:
    PARA CADA h em HORARIO[a]:

```

```

    SUJEITO A Emissao_nao_negativa[a,h]:
        Alpha * pg[a,h]^2
        + Beta * pg[a,h]
        + Gamma
        + Omega * exp(Mu * pg[a,h]) >= 0

    SUJEITO A Emissao_nao_negativa_L2[a,h]:
        Alpha_L2 * pg[a,h]^2
        + Beta_L2 * pg[a,h]
        + Gamma_L2
        + Omega_L2 * exp(Mu_L2 * pg[a,h]) >= 0

// Restrição 2: Emissão zero quando geração é zero
SUJEITO A Emissao_no_zero:
    Gamma + Omega >= 0

SUJEITO A Emissao_no_zero_L2:
    Gamma_L2 + Omega_L2 >= 0

// =====
// FUNÇÕES OBJETIVO
// =====

// Objetivo 1: MSE (Erro Quadrático Médio) - Regressão padrão
FUNCAO_OBJETIVO MSE:
    MINIMIZAR (1 / SOMA_{a em ANO} |HORARIO[a]|)
    * SOMA_{a em ANO} (SOMA_{h em HORARIO[a]} Emissao_horaria[a,h] -
    ↪ emissoes[a])^2

// Objetivo 2: MSE com regularização L2
FUNCAO_OBJETIVO MSE_L2:
    MINIMIZAR (1 / SOMA_{a em ANO} |HORARIO[a]|)
    * SOMA_{a em ANO} (SOMA_{h em HORARIO[a]} Emissao_horaria_L2[a,h] -
    ↪ emissoes[a])^2
    + lambda * (Alpha_L2^2 + Beta_L2^2 + Gamma_L2^2 + Omega_L2^2 +
    ↪ Mu_L2^2)

// =====
// DEFINIÇÃO DOS PROBLEMAS

```

```
// =====
```

```
PROBLEMA Regressao_Estatica:
```

```
  FUNCAO_OBJETIVO: MSE
```

```
  VARIÁVEIS: Emissao_horaria, Alpha, Beta, Gamma, Omega, Mu
```

```
  RESTRIÇÕES: Emissao_ao_negativa, Emissao_no_zero
```

```
PROBLEMA Regressao_Estatica_L2:
```

```
  FUNCAO_OBJETIVO: MSE_L2
```

```
  VARIÁVEIS: Emissao_horaria_L2, Alpha_L2, Beta_L2, Gamma_L2,
```

```
    ↪ Omega_L2, Mu_L2
```

```
  RESTRIÇÕES: Emissao_ao_negativa_L2, Emissao_no_zero_L2
```

```
FIM ALGORITMO
```

Pseudocódigo 4 – Geração das Emissões Sintéticas

```
ALGORITMO geracao_sinteticas
```

```
SAÍDA: Arquivos Parquet salvos em disco (sem retorno explícito)
```

```
INÍCIO
```

```
  // ===== INICIALIZAÇÃO =====
```

```
  horas <- dicionário vazio
```

```
  anos <- listar arquivos no diretório 'IEMA/Tabelas'
```

```
  remover 'desktop.ini' do conjunto
```

```
  para cada arquivo, remover extensão (ex: '.parquet')
```

```
  ordenar anos em ordem crescente
```

```
  // Calcular horas por ano (considerando anos bissextos)
```

```
  PARA CADA ano EM anos:
```

```
    converter ano para inteiro
```

```
    SE ano % 4 == 0 ENTÃO
```

```
      horas[ano] <- 8784    // Ano bissexto (366 dias)
```

```
    SENÃO
```

```
      horas[ano] <- 8760    // Ano normal (365 dias)
```

```
    FIM SE
```

```
FIM PARA
```

```
// ===== PROCESSAMENTO POR USINA =====

PARA CADA usina EM listar_diretorios('Usinas'):

    // Carregar dados de geração da usina
    caminho_pg <- caminho('Usinas', usina, 'Dados Externos',
        ↪ 'ONS.parquet')
    PG <- ler_parquet(caminho_pg)['Geração'] // Array/Series de
        ↪ geração

    pastas <- ['Padrão', 'Regressão L2']

    // ===== CÁLCULO DE EMISSÕES SINTÉTICAS
    ↪ =====

    PARA CADA pasta EM pastas:

        caminho_coeficientes <- caminho('Usinas', usina,
            ↪ 'Minimização',
                                   pasta, 'Emissões',
            ↪ 'Coeficientes.parquet')

        SE arquivo_existe(caminho_coeficientes) ENTÃO

            // Carregar tabela de coeficientes
            tab <- ler_parquet(caminho_coeficientes)

            // Extrair coeficientes da equação
            alpha <- valor de tab onde Coeficientes == 'Alpha'
            beta  <- valor de tab onde Coeficientes == 'Beta'
            gamma <- valor de tab onde Coeficientes == 'Gamma'
            omega <- valor de tab onde Coeficientes == 'Omega'
            mu    <- valor de tab onde Coeficientes == 'Mu'

            // Calcular emissões horárias sintéticas
            // Equação: emissão =  $\alpha \cdot (PG/100)^2 +$ 
            ↪  $\beta \cdot (PG/100) + \gamma +$ 
            ↪  $\omega \cdot e^{(\mu \cdot (PG/100))}$ 
```



```
emissao <- lista vazia

PARA k DE 0 ATÉ comprimento(PG) - 1:
  pg_normalizado <- PG[k] / 100

  termo_quadratico <- alpha * (pg_normalizado^2)
  termo_linear    <- beta * pg_normalizado
  termo_constante <- gamma
  termo_exponencial <- omega * exp(mu *
    ↪ pg_normalizado)

  valor_emissao <- termo_quadratico + termo_linear +
    termo_constante + termo_exponencial

  adicionar arredondar(valor_emissao, 3) em emissao
FIM PARA

// ===== CRIAÇÃO DO DATAFRAME HORÁRIO
↪ =====

horas_expandidas <- lista vazia
anos_expandidos <- lista vazia

// Expandir horas e anos para cada período
PARA CADA ano EM anos:
  inicio <- 0
  num_horas <- horas[ano]

  // Adicionar índices de 0 até num_horas-1
  horas_expandidas.extend(lista de 0 até num_horas-1)

  // Adicionar o ano repetido num_horas vezes
  anos_expandidos.extend([ano] * num_horas)
FIM PARA

// Criar DataFrame com dados horários
df_horario <- DataFrame({
  'Ano': anos_expandidos,
  'Índice': horas_expandidas,
```

```
        'Emissão': emissao
    })

// Salvar arquivo de emissões horárias
caminho_horario <- caminho('Usinas', usina,
    ↪ 'Minimização',
                                pasta, 'Emissões',
                                ↪ 'Horárias.parquet')
salvar_parquet(df_horario, caminho_horario, index=FALSO)

// ===== CÁLCULO DE EMISSÕES ANUAIS
    ↪ =====

// Agrupar por ano e somar emissões
df_anual <-
    ↪ df_horario.agrupar_por('Ano')['Emissão'].somar()
df_anual <- resetar_index(df_anual)

// Carregar emissões reais do IEMA
caminho_iema <- caminho('Usinas', usina, 'Dados
    ↪ Externos', 'IEMA.parquet')
iema <- ler_parquet(caminho_iema)['Emissões']
emissoes_reais <- converter_para_lista(iema)

// Obter emissões anuais sintéticas
emissoes_sinteticas <-
    ↪ df_anual['Emissão'].converter_para_lista()

// Calcular  $K_i = \text{Emissão\_Sintética} / \text{Emissão\_Real}$ 
valores_ki <- lista vazia

PARA j DE 0 ATÉ comprimento(emissoes_sinteticas) - 1:
    ki <- emissoes_sinteticas[j] / emissoes_reais[j]
    adicionar arredondar(ki, 3) em valores_ki
FIM PARA

// Adicionar coluna  $K_i$  ao DataFrame anual
df_anual['K_i'] <- valores_ki
```

```

        // Salvar arquivo de emissões anuais
        caminho_anual <- caminho('Usinas', usina, 'Minimização',
                                pasta, 'Emissões',
                                ↪ 'Anuais.parquet')
        salvar_parquet(df_anual, caminho_anual, index=FALSO)

    FIM SE

FIM PARA

FIM PARA

FIM ALGORITMO

```

Pseudocódigo 5 – Preparação das *Features* da Rede Neural

```

ALGORITMO tratamento_dados
ENTRADA: iema_recente (DataFrame)
SAÍDA: nenhum retorno explícito (salva arquivos em disco)

INÍCIO
    // Definir pastas a processar baseado no tipo de previsão
    pastas <- ['Padrão', 'Regressão L2']

    // Processar cada usina
    PARA CADA usina EM listar_diretorios('Usinas'):

        PARA CADA pasta EM pastas:

            // Carregar dados de geração e emissão
            geracao <- ler_parquet(caminho('Usinas', usina, 'Dados
            ↪ Externos', 'ONS.parquet'))
            emissao <- ler_parquet(caminho('Usinas', usina,
            ↪ 'Minimização', pasta, 'Emissões', 'Horárias.parquet'))

            // Carregar dados unitários (tentando por 'Usina' ou 'CEG')
            TENTAR
                unitarios <- iema_recente[iema_recente['Usina'] ==
                ↪ usina]

```

```
remover coluna 'Usina' de unitarios
resetar índice de unitarios
EXCETO KeyError
    unitarios <- iema_recente[iema_recente['CEG'] == usina]
    remover coluna 'CEG' de unitarios
    resetar índice de unitarios
FIM TENTAR

// Unificar os dados
df <- mesclar(geracao, emissao, how='left')
df <- mesclar(df, unitarios, how='cross')

// == TRANSFORMAÇÃO DAS HORAS EM VALORES CÍCLICOS ==
horas <- dicionário vazio
anos <- listar arquivos em 'IEMA/Tabelas' (excluindo
    ↪ 'desktop.ini')
remover extensão '.parquet' dos nomes
ordenar anos

PARA CADA ano EM anos:
    SE ano % 4 == 0 ENTÃO
        horas[ano] <- 8784
    SENÃO
        horas[ano] <- 8760
    FIM SE

H <- mapear df['Ano'] usando dicionário horas
theta <- 2 * PI * df['Índice'] / H

df['Cosseno Índice'] <- cos(theta)
df['Seno Índice'] <- sin(theta)

// == FEATURES DE TRANSIÇÃO ==
// Duração da transição
transicao_list <- converter (df['Categoria de geração'] ==
    ↪ 0) para inteiro
duracao <- vetor de zeros (mesmo tamanho de transicao_list)
contador <- 0
```

```
PARA i DE 0 ATÉ tamanho(transicao_list) - 1:
  SE transicao_list[i] == 1 ENTÃO
    contador <- contador + 1
  SENÃO
    contador <- 0
  duracao[i] <- contador
df['Duração da Transição'] <- duracao

// Fase da transição
fase <- vetor de zeros (mesmo tamanho de transicao_list)

PARA i DE 0 ATÉ tamanho(transicao_list) - 1:
  SE transicao_list[i] == 1 ENTÃO // em transição
    inicio <- (i == 0) OU (transicao_list[i-1] == 0)
    fim <- (i == tamanho(transicao_list)-1) OU
    ↪ (transicao_list[i+1] == 0)

    SE inicio E fim ENTÃO
      fase[i] <- 1 // transição de uma hora
    SENÃO SE inicio ENTÃO
      fase[i] <- 1 // início
    SENÃO SE fim ENTÃO
      fase[i] <- 3 // fim
    SENÃO
      fase[i] <- 2 // meio
  FIM SE
FIM SE

df['Fase da Transição'] <- fase

// Separar dados em treino, validação e teste
removidas <- ['Delta menos', 'Delta mais', 'Índice']
adicionar 'Ano' a removidas
FIM SE

ano_divisao <- 2023

x_tr <- df[df['Ano'] < ano_divisao]
resetar índice de x_tr
remover colunas removidas de x_tr
```

```
x_val <- df[df['Ano'] == ano_divisao]
resetar índice de x_val
remover colunas removidas de x_val

x_te <- df[df['Ano'] > ano_divisao]
resetar índice de x_te
remover colunas removidas de x_te

// Definir colunas categóricas e numéricas
categoricas <- ['Categoria de geração']
numericas <- ['Geração', 'Duração da Transição', 'Fase da
↳ Transição']

// Configurar transformador de colunas
transf <- ColumnTransformer([
    ('onehot', OneHotEncoder(sparse_output=False,
↳ handle_unknown='ignore'), categoricas),
    ('scaler', StandardScaler(), numericas)
], remainder='passthrough', verbose_feature_names_out=False)

// Aplicar transformações
data_tr <- transf.fit_transform(x_tr)
data_val <- transf.transform(x_val)
data_te <- transf.transform(x_te)

// Criar DataFrames finais
df_tr <- DataFrame(data_tr, colunas =
↳ transf.get_feature_names_out())
df_val <- DataFrame(data_val, colunas =
↳ transf.get_feature_names_out())
df_te <- DataFrame(data_te, colunas =
↳ transf.get_feature_names_out())

// Extrair scaler para coluna 'Geração'
scaler <- transf.named_transformers_['scaler']
idx_pg <- posição de 'Geração' em numericas

df_norm <- DataFrame({
```

```

        'Variável': ['Geração'],
        'Mean': [scaler.mean_[idx_pg]],
        'Std': [scaler.scale_[idx_pg]]
    })

    // Definir diretório de saída
    dir <- caminho('Usinas', usina, 'Minimização', pasta, 'Rede
    ↪ Neural', 'Dados')

    // Criar diretório se não existir
    criar_diretorio(dir, exist_ok=VERDADEIRO)

    // Salvar DataFrames
    salvar_parquet(df_tr, caminho(dir, 'Treino.parquet'),
    ↪ index=FALSO)
    salvar_parquet(df_val, caminho(dir, 'Validação.parquet'),
    ↪ index=FALSO)
    salvar_parquet(df_te, caminho(dir, 'Teste.parquet'),
    ↪ index=FALSO)
    salvar_parquet(df_norm, caminho(dir, 'Scaler
    ↪ Geração.parquet'), index=FALSO)

    FIM PARA
    FIM PARA

    FIM ALGORITMO

```

Pseudocódigo 6 – Treinamento da Rede Neural

```

ALGORITMO treinar_modelo
ENTRADA:
    model (rede neural)
    train_loader (dados de treino)
    val_loader (dados de validação)
    dyn_cols (colunas dinâmicas)
    epochs (inteiro, padrão = 100)
    patience (inteiro, padrão = 50)
    huber_delta (float, padrão = 0.5)
    peso_transicao (float, padrão = 8.0)

```

```
clip_grad (float, padrão = 1.0)
```

SAÍDA:

```
model (modelo treinado)
history (dicionário com histórico de métricas)
```

INÍCIO

```
// Mover modelo para dispositivo (CPU/GPU)
model <- mover_para_device(model, device)

// Configurar otimizador e funções de perda
optimizer <- Adam(model.parameters(), lr=0.001)
criterion_huber <- HuberLoss(delta=huber_delta)
criterion_mae <- L1Loss()
criterion_mse <- MSELoss()

// Configurar scheduler para reduzir learning rate
scheduler <- ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
                               patience=5, min_lr=0.00001)

// Inicializar variáveis de controle
best_val_loss <- infinito
patience_counter <- 0
best_model_state <- None
history <- {
    'loss': [], 'mae': [], 'mse': [],
    'val_loss': [], 'val_mae': [], 'val_mse': []
}

// Loop de épocas
PARA epoch DE 1 ATÉ epochs:

    // === FASE DE TREINAMENTO ===
    model.treino()
    train_loss <- 0.0
    train_mae <- 0.0
    train_mse <- 0.0

    PARA CADA (x_dyn, x_sta, y) EM train_loader:
```



```
// Mover dados para o dispositivo
x_dyn <- mover_para_device(x_dyn, device)
x_sta <- mover_para_device(x_sta, device)
y <- mover_para_device(y, device)
y <- squeeze(y) // Remover dimensões extras

// Zerar gradientes
optimizer.zerar_gradientes()

// Forward pass
output <- model(x_dyn, x_sta)

// Calcular perda base (Huber)
loss_data <- criterion_huber(output, y)

// Aplicar pesos para transições
is_transicao <- (x_sta[:, 0] == 1)
weight <- onde(is_transicao, peso_transicao, 1.0)
loss <- media(loss_data * weight)

// Backward pass
loss.backward()

// Clip gradients para evitar explosão
clip_grad_norm_(model.parameters(), max_norm=clip_grad)

// Atualizar pesos
optimizer.step()

// Acumular métricas
train_loss <- train_loss + loss.item()
train_mae <- train_mae + media(criterion_mae(output,
↪ y)).item()
train_mse <- train_mse + media(criterion_mse(output,
↪ y)).item()

// Calcular médias das métricas de treino
train_loss <- train_loss / tamanho(train_loader)
train_mae <- train_mae / tamanho(train_loader)
```

```
train_mse <- train_mse / tamanho(train_loader)

// === FASE DE VALIDAÇÃO ===
model.avaliacao() // Modo de avaliação (sem dropout, etc.)
val_loss <- 0.0
val_mae <- 0.0
val_mse <- 0.0

// Desabilitar cálculo de gradientes para validação
COM torch.no_grad():
    PARA CADA (x_dyn, x_sta, y) EM val_loader:
        x_dyn <- mover_para_device(x_dyn, device)
        x_sta <- mover_para_device(x_sta, device)
        y <- mover_para_device(y, device)
        y <- squeeze(y)

        output <- model(x_dyn, x_sta)

        loss_val <- media(criterion_huber(output, y))
        val_loss <- val_loss + loss_val.item()
        val_mae <- val_mae + criterion_mae(output, y).item()
        val_mse <- val_mse + criterion_mse(output, y).item()

// Calcular médias das métricas de validação
val_loss <- val_loss / tamanho(val_loader)
val_mae <- val_mae / tamanho(val_loader)
val_mse <- val_mse / tamanho(val_loader)

// Registrar histórico
adicionar train_loss em history['loss']
adicionar train_mae em history['mae']
adicionar train_mse em history['mse']
adicionar val_loss em history['val_loss']
adicionar val_mae em history['val_mae']
adicionar val_mse em history['val_mse']

// Atualizar scheduler baseado na loss de validação
scheduler.step(val_loss)
```

```
// Verificar se é o melhor modelo até agora
SE val_loss < best_val_loss ENTÃO
    best_val_loss <- val_loss
    patience_counter <- 0
    best_model_state <- copiar(model.state_dict())
SENÃO
    patience_counter <- patience_counter + 1

    SE patience_counter >= patience ENTÃO
        INTERROMPER loop // Early stopping
    FIM SE
FIM SE

FIM PARA

// Carregar o melhor modelo encontrado
SE best_model_state NÃO é None ENTÃO
    model.load_state_dict(best_model_state)
FIM SE

RETORNAR model, history

FIM ALGORITMO
```

A Título do Anexo 1

A.1 Seção do anexo 1

XXXXX

A.1.1 Subseção do anexo 1